

Chapter 13

5.A)

```
void capitalize(char str[])
{
    for (int i = 0; str[i] != '\0'; i++)
    {
        str[i] = toupper(str[i]);
    }
}
```

5.B)

```
void capitalize(char *str[])
{
    for (char *i = str; *i != '\0'; i++)
    {
        i* = toupper(i);
    }
}
```

9)

S1 = computers

12)

```
void get_extension(const char *file_name, char *extension) {  
    int len = strlen(file_name);  
    const char *dot = NULL;  
  
    for (int i = len - 1; i >= 0; i--)  
    {  
        dot = (file_name[i] == '.') ? &file_name[i + 1] : dot;  
        if (dot) break;  
    }  
  
    strcpy(extension, dot ? dot : "");  
}
```

Chapter 14

1.A) #define CUBE(x) ((x) * (x) * (x))

1.B) #define REMAINDER_BY_4(n) ((n) % 4)

1.C) #define LESS_THAN_100(x, y) (((x) * (y)) < 100 ? 1 : 0)

1.D) Potential issues with the cube macro is that if an argument like ++x or x++ are passed, they may be evaluated multiple times, causing unintended behavior. Also an issue for all the macros are complex expressions without parenthesis, which can lead to incorrect results due to how the operations are processed.

4) Issues with both of the macros include processing complex expressions incorrectly due to lack of parentheses and potential truncation errors when performing integer division. For example, without parentheses, the expression $4 + 5 / 2$ would calculate as $4 + (5 / 2)$, resulting in $4 + 2 = 6$. Adding parentheses to make it $(4 + 5) / 2$ correctly calculates the average as 4.5, but if both operands are integers, the division may truncate the result, yielding 4 instead of 4.5. Using floating-point numbers can help avoid this truncation.

13.A)

```
void f(void);
```

```
int main(void)
```

```
{
```

```
    f();
```

```
    return 0;
```

```
}
```

```
void f(void)
```

```
{
```

```
    printf("N is %d\n", 100);
```

```
}
```

13.B) $n = 100$

Chapter 15

4.A)

Output if DEBUG is defined

Value of n: 1

Value of n: 2

Value of n: 3

Value of n: 3

Value of n: 1

4.B)

Output if DEBUG is not defined

4.C) The difference in outputs stems from the presence or absence of the DEBUG macro. In part (a), DEBUG is defined, allowing the PRINT_DEBUG macro to expand to printf statements that display the values of variables. In part (b), without DEBUG, the PRINT_DEBUG macro expands to nothing, effectively making those calls do nothing and only outputting the alternative message.

4.D) Yes, it is necessary for the DEBUG macro to be defined before including debug.h. The macro definitions inside debug.h rely on whether DEBUG is defined at the time the header is processed. If DEBUG is not defined before including the header, the PRINT_DEBUG macro will not function as intended, and all calls to it will be effectively ignored, leading to no output for those statements.

5)

Makefile for demo program

Compiler

CC = gcc

Compiler flags

CFLAGS = -Wall -g

Source files

SRCS = main.c f1.c f2.c

Header files

HEADERS = f1.h f2.h

Object files (replace .c with .o)

OBJS = \$(SRCS:.c=.o)

Executable name

EXEC = demo

Default target

all: \$(EXEC)

Rule to link object files to create the executable

\$(EXEC): \$(OBJS)

```
$(CC) $(CFLAGS) -o $@ $^
```

```
# Rule to compile source files into object files
```

```
%.o: %.c $(HEADERS)
```

```
$(CC) $(CFLAGS) -c $<
```

```
# Clean rule to remove object files and the executable
```

```
clean:
```

```
rm -f $(OBJS) $(EXEC)
```

```
# Phony targets
```

```
.PHONY: all clean
```

Chapter 16

2.A)

```
#include <stdio.h>
```

```
// Define a structure for complex numbers
```

```
struct Complex {
```

```
    double real;
```

```
    double imaginary;
```

```
};
```

```
// Declare structure variables
```

```
struct Complex c1, c2, c3;
```

2.B)

// Declare and initialize structure variables

struct Complex c1 = {0.0, 1.0}; // c1's members initialized to 0.0 and 1.0

struct Complex c2 = {1.0, 0.0}; // c2's members initialized to 1.0 and 0.0

struct Complex c3; // c3 is uninitialized

2.C)

One statement

c1 = c2; // This copies all members from c2 to c1

two statements

c1.real = c2.real; // Copy real part

c1.imaginary = c2.imaginary; // Copy imaginary part

2.D)

c3.real = c1.real + c2.real; // Add real parts

c3.imaginary = c1.imaginary + c2.imaginary; // Add imaginary parts

6)

#include <stdio.h>

// Define the time structure

struct time {

int hours;

int minutes;

int seconds;

```
};
```

```
// Function to split total seconds into hours, minutes, and seconds
```

```
struct time split_time(long total_seconds) {
```

```
    struct time t; // Create a time structure to hold the result
```

```
    // Calculate hours, minutes, and seconds
```

```
    t.hours = (total_seconds / 3600) % 24; // Get hours in the range of 0-23
```

```
    t.minutes = (total_seconds / 60) % 60; // Get minutes in the range of 0-59
```

```
    t.seconds = total_seconds % 60;    // Get seconds in the range of 0-59
```

```
    return t; // Return the time structure
```

```
}
```

```
int main() {
```

```
    long total_seconds = 3661; // Example: 1 hour, 1 minute, and 1 second
```

```
    struct time current_time = split_time(total_seconds);
```

```
    // Print the result
```

```
    printf("Time: %02d:%02d:%02d\n", current_time.hours, current_time.minutes,  
current_time.seconds);
```

```
    return 0;
```

```
}
```


8.A)

```
#include <stdio.h>
```

```
struct color {  
    int red;  
    int green;  
    int blue;  
};
```

```
// Declare a const variable MAGENTA
```

```
const struct color MAGENTA = {255, 0, 255};
```

8.B)

```
#include <stdio.h>
```

```
struct color {  
    int red;  
    int green;  
    int blue;  
};
```

```
// Declare a const variable MAGENTA using a designated initializer
```

```
const struct color MAGENTA = {.red = 255, .blue = 255}; // green defaults to 0
```

10.A)

```
int compute_area(struct rectangle r)
{
    int width = r.lower_right.x - r.upper_left.x;
    int height = r.lower_right.y - r.upper_left.y;
    return width * height;
}
```

10.B)

```
struct point compute_center(struct rectangle r)
{
    struct point center;
    center.x = (r.upper_left.x + r.lower_right.x) / 2;
    center.y = (r.upper_left.y + r.lower_right.y) / 2;
    return center;
}
```

10.C)

```
struct rectangle move_rectangle(struct rectangle r, int x, int y)
{
    r.upper_left.x += x;
    r.upper_left.y += y;
    r.lower_right.x += x;
    r.lower_right.y += y;
    return r;
}
```

10.D)

```
int is_point_within_rectangle(struct rectangle r, struct point p)
{
    return (p.x >= r.upper_left.x && p.x <= r.lower_right.x) &&
        (p.y >= r.upper_left.y && p.y <= r.lower_right.y);
}
```